Name- Bhakare Purva Shivaji

Roll no- 08

Practical no- 11

```
import os
import zipfile
import glob
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier
from sklearn.metrics import accuracy_score

# 1. Load dataset from ZIP, or use built-in dataset
zip_path = r"C:\Users\PVGIT\Downloads\archive (19).zip"
extract_path = r"C:\Users\PVGIT\Downloads\ml_dataset"
df = None
try:
    os.makedirs(extract_path, exist_ok=True)
    with zipfile.ZipFile(zip_path, 'r') as z:
        z.extractall(extract_path)
    files = glob.glob(extract_path + "/**/*", recursive=True)
    files = [f for f in files if os.path.isfile(f)]

    for f in files:
        if f.lower().endswith('.csv'):
            df = pd.read_csv(f)
```

```
            break
        elif f.lower().endswith(('.xlsx', '.xls')):
            df = pd.read_excel(f)
            break
except Exception as e:
    print("ZIP loading issue:", e)
if df is not None and df.shape[1] >= 2:
    df = df.dropna()
    X = pd.get_dummies(df.iloc[:, :-1])
    y = df.iloc[:, -1]
    y = pd.factorize(y)[0]
    print("Dataset loaded from ZIP.")
else:
    data = load_breast_cancer()
    X = pd.DataFrame(data.data)
    y = data.target
    print("Using built-in Breast Cancer dataset.")
# Remove invalid values
X = X.replace([np.inf, -np.inf], np.nan)
valid = X.notna().all(axis=1)
X = X.loc[valid]
y = np.asarray(y)[valid.to_numpy()]
# 2. Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 3. Define models and parameter grids
models = {
    "SVM": (
        make_pipeline(StandardScaler(), SVC()),
```

```
        {"svc__C": [0.1, 1, 10],
         "svc__kernel": ["linear", "rbf"]}
    ),
    "Random Forest": (
        RandomForestClassifier(random_state=42),
        {"n_estimators": [50, 100],
         "max_depth": [None, 10]}
    ),
    "AdaBoost": (
        AdaBoostClassifier(random_state=42),
        {"n_estimators": [50, 100],
         "learning_rate": [0.5, 1.0]}
    )}
# 4. Train, cross-validate and optimize
print("\nMODEL PERFORMANCE COMPARISON")
print("-" * 65)
for name, (model, params) in models.items():
    cv_before = cross_val_score(
        model, X_train, y_train, cv=5, scoring="accuracy"
    ).mean()
    model.fit(X_train, y_train)
    before = accuracy_score(y_test, model.predict(X_test))

    grid = GridSearchCV(
        model, params, cv=5, scoring="accuracy", n_jobs=-1
    )
    grid.fit(X_train, y_train)
    after = accuracy_score(
        y_test, grid.best_estimator_.predict(X_test)
    )
    print("\nModel:", name)
```

```
    print(f"Before Optimization - CV Accuracy: {cv_before:.4f}")
    print(f"Before Optimization - Test Accuracy: {before:.4f}")
    print(f"After Optimization - CV Accuracy: {grid.best_score_:.4f}")
    print(f"After Optimization - Test Accuracy: {after:.4f}")
    print("Best Parameters:", grid.best_params_)
print("\nConclusion: Compare test accuracy before and after")
print("Grid Search to identify the best-performing model.")
```

OUTPUT-

```
Using built-in Breast Cancer dataset.
MODEL PERFORMANCE COMPARISON
----------------------------------------------------------------

Model: SVM
Before Optimization - CV Accuracy: 0.9714
Before Optimization - Test Accuracy: 0.9825
After Optimization - CV Accuracy: 0.9780
After Optimization - Test Accuracy: 0.9825
Best Parameters: {'svc__C': 0.1, 'svc__kernel': 'linear'}

Model: Random Forest
Before Optimization - CV Accuracy: 0.9538
Before Optimization - Test Accuracy: 0.9561
After Optimization - CV Accuracy: 0.9538
After Optimization - Test Accuracy: 0.9561
Best Parameters: {'max_depth': None, 'n_estimators': 100}

Model: AdaBoost
Before Optimization - CV Accuracy: 0.9692
Before Optimization - Test Accuracy: 0.9561
After Optimization - CV Accuracy: 0.9780
```

After Optimization - Test Accuracy: 0.9561

Best Parameters: {'learning_rate': 0.5, 'n_estimators': 100}

Conclusion: Compare test accuracy before and after

Grid Search to identify the best-performing model.